

LinkedList

June 24, 2026

0.1 Linked List

```
[2]: ## Define a linked list class in python
class LinkedList:
    def __init__(self, val = 0, next = None):
        self.val = val
        self.next = next
```

```
[3]: ## Produce an instance
node3 = LinkedList(3)
node2 = LinkedList(2, node3)
node1 = LinkedList(1, node2)
```

```
[4]: # This gives the value
print(node1.val)

# This gives the memory address of the next LinkedList after node1 (which would
↪ be node2)
print(node1.next)
```

1

```
<__main__.LinkedList object at 0x106bf96d0>
```

So why do we care about Linked Lists at all? Surely it does the same thing as arrays?

0.1.1 Insertion/Deletion

Compared to arrays, linked lists has $O(1)$ insertion/deletion at front, compared to $O(n)$ for arrays.

```
[5]: # Consider array insertion
x = [1,2,3]

# Insert 0 at front would be O(n) as it would have to move the pointers/
↪ references.
x.insert(0, 0)
print(x)

# For linked lists, this would be O(1)
head = LinkedList(0, next=node1)
```

```
print(head.val)
print(head.next.val)
```

[0, 1, 2, 3]

0

1

```
[ ]: # Consider array deletion
```

```
# Deleting the first element for an array looks like this,  $O(n)$ 
```

```
x.pop(0)
```

```
print(x)
```

```
# Whereas delete the first element for a linked list takes  $O(1)$ , any other  
↳ nodes are left the same
```

```
head = head.next
```

[1, 2, 3]

0.1.2 Disadvantages

- You can't access the nth element in $O(1)$.
- Extra memory: both value and next are stored.